Upside-down Hachiware



วันหนึ่งชิคาวะออกมาเดินเล่นกับฮาจิวาเระตามปกติ แต่ระหว่างที่กำลังคุยกันอยู่ ชิคาวะกลับสังเกตเห็นบางอย่างผิดปกติ เพราะร่างกายของฮาจิวาเระไม่ได้อยู่ในสภาพเดิมอีกต่อไป หัวของฮาจิวาเระกลับไปอยู่ด้านล่าง ในขณะที่เท้ากลับไปอยู่ด้านบน

ฮาจิวาเระเองก็ไม่รู้ว่าเกิดอะไรขึ้น รู้เพียงว่าก่อนหน้านี้ได้ไปสัมผัสป้ายประหลาดจำนวน N ใบที่วางเรียงกันอยู่ ป้ายแต่ละใบมีตัวเลขกำกับอยู่ และหลังจากสัมผัสป้ายเหล่านั้น ร่างกายของฮาจิวาเระก็กลับด้านทันที

ชิคาวะพบข้อความกำกับอยู่ข้างป้ายว่า หากต้องการทำให้ฮาจิวาเระกลับมาเป็นปกติ จะต้องจัดเรียงป้ายตัวเลขจากค่าน้อยไปหาค่ามากตามวิธีที่กำหนดไว้เท่านั้น

กำหนดให้มีป้ายจำนวน N ใบ เรียงจากซ้ายไปขวา โดยป้ายใบที่ i+1 มีค่า a[i] ในแต่ละรอบ ชิคาวะจะเริ่มตรวจป้ายจากทางซ้ายสุด และเปรียบเทียบป้ายที่อยู่ติดกันทีละคู่จากซ้ายไปขวา เริ่มจากตำแหน่งที่ 0 และ 1 จากนั้นตำแหน่งที่ 1 และ 2 และทำเช่นนี้ต่อไป

หากป้ายทางซ้ายมีค่ามากกว่าป้ายทางขวา ชิคาวะจะสลับป้ายทั้งสองใบทันที ก่อนจะไปพิจารณาคู่ถัดไป เมื่อจบรอบแรก ป้ายที่มีค่ามากที่สุดจะไปอยู่ทางขวาสุดแล้ว ดังนั้นในรอบต่อไปจึงไม่จำเป็นต้องพิจารณาป้ายใบนั้นอีก และทุกๆ รอบหลังจากนั้น จำนวนป้ายที่ต้องนำมาเปรียบเทียบจะลดลงครั้งละหนึ่งใบ

อย่างไรก็ตาม เวทมนตร์ของป้ายจะทำงานได้เพียงช่วงเวลาหนึ่ง ชิคาวะจึงมีเวลาจัดเรียงป้ายตามขั้นตอนดังกล่าวเพียง K รอบเท่านั้น คุณจะต้องช่วยชิคาวะหาว่า เมื่อทำครบ K รอบแล้ว ป้ายจะอยู่ในลำดับใด และเกิดการสลับป้ายทั้งหมดกี่ครั้ง

งานของคุณ (Your Task)

เขียนฟังก์ชันที่มีประสิทธิภาพเพื่อจำลองการจัดเรียงป้ายของชิคาวะจำนวน K รอบตามขั้นตอนที่กำหนดไว้ข้างต้น เมื่อทำงานครบ K รอบแล้ว อาเรย์ a จะต้องมีค่าเหมือนกับลำดับของป้ายหลังจากชิคาวะดำเนินการครบ K รอบ และฟังก์ชันจะต้องคืนค่าจำนวนครั้งทั้งหมดที่มีการสลับป้ายเกิดขึ้น ในรอบที่ i โดยเริ่มนับรอบจาก 1 จะมีการเปรียบเทียบป้ายตามลำดับดังนี้

$$(a[0], a[1]), (a[1], a[2]), \ldots, (a[N-i-1], a[N-i])$$

หากในการเปรียบเทียบตำแหน่ง j พบว่า

$$a[j] > a[j+1]$$

จะต้องสลับค่าของ a[j] และ a[j+1] ทันที ก่อนที่จะพิจารณาป้ายคู่ถัดไป

ข้อนี้มีรูปแบบการเขียนโปรแกรมที่แตกต่างจากรูปแบบปรกติ

โปรดดูส่วนท้ายของโจทย์ข้อนี้เกี่ยวกับวิธีการใช้งาน

รายละเอียดของโปรแกรม

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

```
long long hachiware_bubble(long long a[], int N, int K)
```

โดยที่พารามิเตอร์ต่าง ๆ คือ

- a เป็นอาเรย์จำนวน N ช่อง แสดงค่าของป้ายที่เรียงจากซ้ายไปขวา
- N เป็นจำนวนป้ายทั้งหมด
- K เป็นจำนวนรอบที่ชิคาวะจะดำเนินการจัดเรียงป้าย
- เมื่อฟังก์ชันทำงานเสร็จ อาเรย์ a จะต้องถูกแก้ไขให้ตรงกับลำดับของป้ายหลังจากดำเนินการครบ K รอบ
- ฟังก์ชันจะต้องคืนค่าเป็นจำนวนเต็มชนิด long long เพื่อแสดงจำนวนครั้งทั้งหมดที่มีการสลับป้ายเกิดขึ้น

ขอบเขตข้อมูล

- $1 \le N \le 5000$
- $0 \le K \le N-1$
- $-10^9 \le a[i] \le 10^9$

ตัวอย่าง

ลองพิจารณาการเรียกใช้งานต่อไปนี้

กรณีตัวอย่างที่ 1

N: 5

K: 2

a: [5, 1, 4, 2, 3]

return: 6

a: [1, 2, 3, 4, 5]

ผลลัพธ์จะได้เป็น 6 เนื่องจากในรอบแรก ชิคาวะเริ่มต้นจากลำดับ 5 1 4 2 3 เปรียบเทียบ 5 กับ 1 และสลับ จะได้ 1 5 4 2 3 จากนั้นเปรียบเทียบ 5 กับ 4 และสลับ 1 4 5 2 3 เปรียบเทียบ 5 กับ 2 และสลับ 1 4 2 5 3 และเปรียบเทียบ 5 กับ 3 และสลับ 1 4 2 3 5 ดังนั้นรอบแรกเกิดการสลับทั้งหมด 4 ครั้ง

ในรอบที่สอง ไม่จำเป็นต้องนำป้าย 5 ที่อยู่ตำแหน่งสุดท้ายมาพิจารณาอีก หลังจากเปรียบเทียบป้ายที่เหลือจากซ้ายไปขวาจะได้ 1 2 3 4 5 โดยเกิดการสลับเพิ่มอีก 2 ครั้ง ดังนั้นหลังจากครบ 2 รอบ มีการสลับทั้งหมด 4 + 2 = 6 ครั้ง

กรณีตัวอย่างที่ 2

N: 5

K: 1

a: [5, 4, 3, 2, 1]

return: 4

a: [4, 3, 2, 1, 5]

ผลลัพธ์จะได้เป็น 4 เนื่องจากในรอบแรก ป้าย 5 จะถูกสลับกับป้ายทางขวาทีละใบจนกระทั่งไปอยู่ตำแหน่งสุดท้าย จึงได้ลำดับ 4 3 2 1 5 โดยเกิดการสลับทั้งหมด 4 ครั้ง สังเกตว่าป้ายทั้งหมดยังไม่ได้ถูกเรียงจากน้อยไปมาก เนื่องจากชิคาวะดำเนินการเพียง 1 รอบเท่านั้น

กรณีตัวอย่างที่ 3

N: 6

K: 3

a: [1, 2, 3, 4, 5, 6]

return: 0

a: [1, 2, 3, 4, 5, 6]

ผลลัพธ์จะได้เป็น 0 เนื่องจากป้ายทั้งหมดถูกเรียงจากน้อยไปมากอยู่แล้ว ทุกครั้งที่ชิคาวะเปรียบเทียบป้ายที่อยู่ติดกันจึงไม่มีการสลับเกิดขึ้น หลังจากทำครบ 3 รอบ ลำดับของป้ายจึงยังคงเดิม และจำนวนครั้งที่มีการสลับเท่ากับ 0

กรณีตัวอย่างที่ 4

N: 4

K: 0

a: [4, 1, 3, 2]

return: 0

a: [4, 1, 3, 2]

เนื่องจาก K=0 ชิคาวะจะไม่ดำเนินการจัดเรียงป้ายแม้แต่รอบเดียว ลำดับของป้ายจึงไม่เปลี่ยนแปลง และไม่มีการสลับเกิดขึ้น

ข้อกำหนด

ระยะเวลาสูงสุดที่ใช้ในการประมวลผล	1 วินาที
หน่วยความจำสูงสุดที่ใช้ในการประมวลผล	16 MB
คะแนนสูงสุดของโจทย์	100 คะแนน

ข้อมูลเพิ่มเติมเกี่ยวกับชุดทดสอบ

ข้อมูลแนะนำที่เกี่ยวข้องกับชุดทดสอบ มีดังนี้

กลุ่มชุดทดสอบที่	คะแนนสูงสุดของกลุ่มชุดทดสอบนี้	เงื่อนไข
1	5%	$N \leq 10$
2	10%	$N \leq 100$ และ $K = 1$
3	10%	$N \leq 100$ และ $a[i] \leq a[i+1]$ สำหรับทุกตำแหน่งที่เป็นไปได้
4	15%	$N \leq 500$
5	20%	$N \leq 1{,}000$
6	20%	$N \leq 3{,}000$
7	20%	ไม่มีเงื่อนไขเพิ่มเติม

คำแนะนำในการเขียนโปรแกรม

สำหรับข้อนี้วิธีการเขียนโปรแกรมและส่งโปรแกรมเข้าสู่ระบบเกรดเดอร์ (Grader) จะเป็นรูปแบบที่แตกต่างจากรูปแบบทั่วไปของการเขียนโปรแกรม ที่คุณจะต้องเขียนโปรแกรมในส่วนรับข้อมูลนำเข้าและส่วนแสดงผล โดยในข้อนี้คุณจะต้องเขียนโปรแกรมเฉพาะส่วนการคำนวณที่จำเป็นลงในฟังก์ชันที่กำหนดให้โดยข้อมูลนำเข้าต่าง ๆ จะสามารถใช้ได้ผ่านพารามิเตอร์ของฟังก์ชัน และผลการคำนวณจะต้องส่งกลับผ่านฟังก์ชันด้วยเช่นกัน โดยที่คุณไม่ต้องเขียนโปรแกรมในส่วนรับข้อมูลนำเข้า และไม่ต้องเขียนส่วนแสดงผล

คุณจะได้รับไฟล์ต่าง ๆ 7 ไฟล์ดังนี้ คุณจะต้องแก้ไขไฟล์เดียวตามที่โจทย์กำหนด และส่งไฟล์ดังกล่าวเข้าสู่ระบบ

- grader.cpp เป็นไฟล์หลักที่ทำหน้าที่รับข้อมูลนำเข้าและส่งออก
- C1T44.h เป็นไฟล์ส่วนหัวที่ใช้สำหรับประกาศฟังก์ชันที่กำหนดให้
- C1T44.cpp เป็นไฟล์ที่คุณจะต้องเขียนโปรแกรมคำนวณต่าง ๆ และเป็นไฟล์ที่คุณส่งเข้าสู่ระบบ (ในไฟล์ C1T44.cpp ห้ามมีฟังก์ชัน main() แต่มีฟังก์ชันอื่น ๆ ได้)
- run_cpp.sh เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการรันโปรแกรมทั้งหมดได้บนระบบ Linux และ macOS
- compile_cpp.sh เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการคอมไพล์โปรแกรมได้บนระบบ Linux และ macOS
- run_cpp.bat เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการรันโปรแกรมทั้งหมดได้บนระบบ Windows
- compile_cpp.bat เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการคอมไพล์โปรแกรมได้บนระบบ Windows

คุณจะได้รับไฟล์ .zip ซึ่งสามารถดาวน์โหลดได้จากระบบโดยในไฟล์ดังกล่าวเมื่อขยาย (extract) ออกมาแล้วจะมีไฟล์ทั้ง 7 อยู่ คุณควรใช้ text editor หรือ IDE ในการแก้ไขไฟล์ C1T44.cpp และทำการคอมไพล์หรือรันโปรแกรมผ่านการเรียกใช้ฟังก์ชัน compile_cpp.sh หรือ run_cpp.sh สำหรับระบบ Linux และ macOS หรือ compile_cpp.bat หรือ run_cpp.bat สำหรับระบบ Windows

เมื่อคุณสามารถเรียกไฟล์ run_cpp.sh หรือ run_cpp.bat เพื่อทำการรันโปรแกรม โปรแกรมจะทำงานตามที่ได้ระบุไว้ในไฟล์ grader.cpp ซึ่งหัวข้อ “คำอธิบายไฟล์เกรดเดอร์” อธิบายถึงการทำงานของไฟล์ดังกล่าว